รายงานโครงงาน

เรื่อง การพัฒนาเว็บแอปพลิเคชันคำนวณทางคณิตศาสตร์ด้วยการเขียนโปรแกรมเชิงวัตถุ

FastNumeric

Numerical Method Calculator

(โปรแกรมคำนวณวิธีเชิงตัวเลขพร้อมแสดงขั้นตอนการคำนวณ)

จัดทำโดย

..... รหัสนักศึกษา

.....

..... รหัสนักศึกษา

.....

เสนอ

สาขาวิชา ..... คณะ

.....

รายวิชา ..... ภาคเรียนที่ ..... ปีการศึกษา

.....

มหาวิทยาลัย.....

บทที่ 1
ที่มาและความสำคัญ

1.1 ที่มาและความสำคัญ

ในการเรียนวิชาวิธีเชิงตัวเลข (Numerical Methods) ผู้เรียนต้องคำนวณคำตอบของปัญหาทางคณิตศาสตร์หลายประเภท เช่น การหารากของสมการ การแก้ระบบสมการเชิงเส้น การแทรกและการคาดคะเนค่า ตลอดจนการหาพื้นที่ใต้เส้นโค้งและอนุพันธ์เชิงตัวเลข ซึ่งแต่ละวิธีต้องทำการคำนวณซ้ำ ๆ จำนวนมาก ถ้าทำด้วยมือจะเสียเวลามากและเกิดข้อผิดพลาดได้ง่าย นอกจากนี้ผู้เรียนยังควรเข้าใจ "ขั้นตอนการคำนวณ" ของแต่ละวิธี ไม่ใช่แค่คำตอบสุดท้ายเท่านั้น

ผู้จัดทำจึงพัฒนาโครงงาน **FastNumeric** เป็นเว็บแอปพลิเคชันสำหรับคำนวณวิธีเชิงตัวเลข จำนวน 6 หมวด 11 วิธี โดยโปรแกรมไม่เพียงแสดงคำตอบสุดท้าย แต่แสดง "ขั้นตอนการคำนวณ" และ "ตารางการทำซ้ำ (Iteration Table)" ครบทุกขั้นตอน ผู้ใช้สามารถตรวจสอบการคำนวณย้อนกลับได้ทีละรอบ เหมาะกับการใช้ตรวจคำตอบและทบทวนวิธีทำของผู้เรียน

ด้านการพัฒนา โปรแกรมถูกออกแบบด้วยหลัก**การเขียนโปรแกรมเชิงวัตถุ (Object-Oriented Programming: OOP)** อย่างเต็มรูปแบบ ทั้งการซ่อนรายละเอียด (Encapsulation) การสืบทอดคุณสมบัติ (Inheritance) ความเป็นนามธรรม (Abstraction) รูปแบบพฤติกรรมหลากหลาย (Polymorphism) และการประกอบวัตถุ (Composition) เพื่อให้ระบบแยกชั้นความรับผิดชอบชัดเจน เพิ่มวิธีคำนวณใหม่ได้โดยไม่ต้องแก้โค้ดเดิม

1.2 ประเภทของโครงการ

โครงการนี้เป็นโครงการประเภท**พัฒนาซอฟต์แวร์ประยุกต์ (Application Development)** เป็นเว็บแอปพลิเคชันที่ทำงานบนเว็บเบราว์เซอร์ ประกอบด้วยส่วนติดต่อผู้ใช้ (Frontend), ส่วนประมวลผลและเก็บข้อมูล (Backend +

Database) และสามารถติดตั้งทำงานบนเซิร์ฟเวอร์ได้ด้วยเทคโนโลยีคอนเทนเนอร์ (Docker)

1.3 ประโยชน์ของโครงการ

1. คำนวณวิธีเชิงตัวเลขได้ถูกต้อง รวดเร็ว ครอบคลุม 6 หมวด 11 วิธีที่ใช้เรียนในรายวิชาวิธีเชิงตัวเลข
2. แสดงขั้นตอนการคำนวณและตารางการทำซ้ำทุกวิธี ช่วยให้ผู้เรียนเข้าใจกลไกของอัลกอริทึม ไม่ใช่แค่คำตอบ
3. ตรวจคำตอบการบ้านหรือการคำนวณด้วยมือได้รวดเร็ว ลดข้อผิดพลาดจากการคำนวณซ้ำ ๆ
4. บันทึกประวัติการคำนวณลงฐานข้อมูล นับจำนวนครั้งที่คำนวณโจทย์เดิม และสรุปสถิติให้เห็นภาพรวม
5. เป็นตัวอย่างการนำหลัก OOP ไปใช้กับโจทย์จริงด้านคณิตศาสตร์ เช่นการแยกชั้นอัลกอริทึมออกจากการแสดงผล

บทที่ 1 (ต่อ)
ขอบเขตของโครงการ

1.4 ขอบเขตของโครงการ

1.4.1 คุณสมบัติทางฟังก์ชัน (Functional Requirements)

หมวด	วิธีคำนวณ	คุณสมบัติที่พัฒนา
Root of Equation	Bisection, Newton-Raphson	หารากของสมการ f(x) = 0 แสดงตารางการทำซ้ำ ตรวจเงื่อนไข f(XL)·f(XU) < 0 และเงื่อนไขความลู่เข้า
Linear Algebra	Gauss Elimination	แก้ระบบสมการเชิงเส้น n ตัวแปร (n = 2–6) ด้วยการตัดตัวแปรแบบ Partial Pivoting ตรวจเมทริกซ์เอกฐน
Interpolation	Linear, Lagrange	หาค่าระหว่างจุดจากตาราง (x, y) ตรวจว่า x อยู่ในช่วงข้อมูลหรือไม่
Extrapolation	Linear	คาดคะเนค่านอกช่วงข้อมูล ตรวจว่า x ต้องอยู่นอกช่วงจริง
Integration	Trapezoidal, Simpson's 1/3	หาพื้นที่ใต้เส้นโค้งจากตารางจุด ตรวจว่า Simpson ต้องมีจำนวนช่วงเป็นเลขคู่ รองรับกรณี a > b
Differentiation	Forward, Backward, Central	หาอนุพันธ์เชิงตัวเลขจากตารางจุด ตรวจว่าจุดที่เลือกมีเพื่อนบ้านครบตามสูตร

1.4.2 คุณสมบัติเชิงระบบ

- กรอกสมการ f(x) ได้โดยตรง เช่น `x^3 - x - 2` โปรแกรมแยกวิเคราะห์สมการเองโดยไม่ใช้ eval()
- กำหนดค่า Tolerance และ Max Iterations ได้ พร้อมค่าเริ่มต้นที่เหมาะสม

- จัดการข้อผิดพลาดเป็นภาษาไทยที่อธิบายสาเหตุ เช่น หารด้วยศูนย์, เมทริกซ์เอกฐน, ไม่ลู่เข้า
- บันทึกประวัติการคำนวณลง SQLite นับโจทย์ซ้ำ (หมวด + วิธี + อินพุตเดิม) ลบรายการได้
- หน้า Dashboard สรุปสถิติ: จำนวนโจทย์, จำนวนครั้งที่คำนวณ, สัดส่วนตามหมวด, โจทย์ยอดนิยม
- ติดตั้งด้วย Docker Compose พร้อม health check และ named volume สำหรับเก็บฐานข้อมูลถาวร

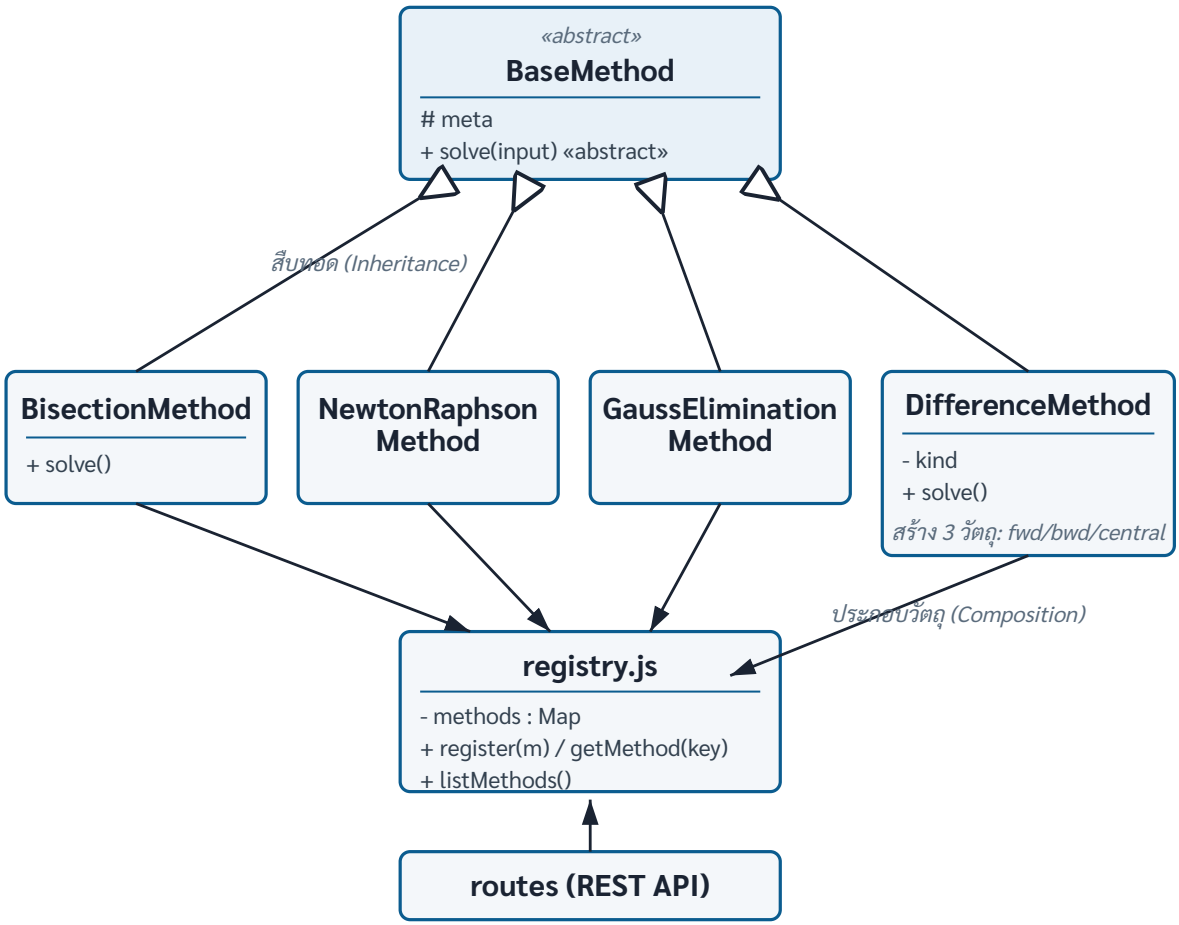
1.4.3 ตารางเวลาการดำเนินโครงการ

กิจกรรม	สัปดาห์ 1	สัปดาห์ 2	สัปดาห์ 3	สัปดาห์ 4	สัปดาห์ 5	สัปดาห์ 6
ศึกษาโจทย์และวิธีเชิงตัวเลขทั้ง 11 วิธี	✓					
ออกแบบโครงสร้างคลาส (OOP) และ REST API		✓				
พัฒนาคลาสอัลกอริทึม + Expression Parser		✓	✓			
พัฒนาฐานข้อมูลและระบบประวัติการคำนวณ			✓	✓		
พัฒนาส่วนติดต่อผู้ใช้ (React + Tailwind)			✓	✓		
เขียนชุดทดสอบและตรวจสอบผลลัพธ์				✓	✓	
จัดทำ Docker / เอกสาร / รายงาน					✓	✓

บทที่ 2
การพัฒนาโครงงาน

2.1 แผนภาพคลาส (Class Diagram)

โครงสร้างคลาสหลักของระบบออกแบบตามหลักการแบ่งชั้น โดยคลาสนามธรรม **BaseMethod** เป็นแม่ของคลาสวิธีคำนวณทั้ง 11 วิธี และมี **registry.js** เป็นศูนย์กลางประกอบ (compose) วัตถุทุกวิธีเข้าด้วยกัน



รูปที่ 2.1 แผนภาพคลาสของระบบ FastNumeric (ส่วนอัลกอริทึม)

2.2 คลาสหลักของระบบ

คลาส / ไฟล์	หน้าที่
BaseMethod	คลาสนามธรรมของทุกวิธีคำนวณ เก็บข้อมูล meta (key, หมวด, ชื่อ, นิยามฟิลด์อินพุต) และประกาศเมธอดนามธรรม solve() ให้

	คลาสลูกไป implement
roots.js / linear.js / interpolation.js / extrapolation.js / integration.js / differentiation.js	คลาสวิธีคำนวณจริง 11 วิธี สืบทอดจาก BaseMethod แต่ละคลาสรับผิดชอบเฉพาะคณิตศาสตร์ของตัวเอง
registry.js	ศูนย์กลางลงทะเบียนวัตถุวิธีคำนวณทั้งหมดใน Map พร้อมเมธอด listMethods() สำหรับสร้าง UI อัตโนมัติ
expression/parser.js	แปลงสมการ f(x) จากสตริงเป็นฟังก์ชันด้วย Tokenizer + Recursive Descent Parser (ไม่ใช้ eval)
validation/index.js	ด่านตรวจอินพุตกลาง (asFloat, asInt, asPoints, asMatrix, asExpression) โยน ValidationError พร้อมข้อความไทย
database/db.js + history.js	ชั้นเข้าถึงข้อมูล: ตาราง problem + calculation ใน SQLite นับโจทย์ซ้ำด้วย input hash
routes/index.js	REST API: /methods, /calculate, /history, /history/:id, /stats
client (React)	MethodInput สร้างฟอร์มอัตโนมัติจาก meta.fields, ResultView แสดงคำตอบ/ขั้นตอน/ตาราง

2.3 รูปแบบการพัฒนาและเทคโนโลยีที่ใช้

ส่วนของระบบ	เทคโนโลยี
Frontend	React 19 + Vite + Tailwind CSS (JavaScript)
Backend	Node.js + Express (REST API)
ฐานข้อมูล	SQLite ผ่าน node:sqlite (DatabaseSync ในตัว Node.js)
การติดตั้ง	Docker + Docker Compose (multi-stage build, Nginx, health check, named volume)

ระบบแบ่งการทำงานเป็น 3 ชั้นที่แยกกันชัดเจน: **ชั้นรับข้อมูล** (ฟอร์ม React สร้างจาก meta ของแต่ละวิธี), **ชั้นอัลกอริทึม** (คลาสวิธีคำนวณสืบทอด BaseMethod) และ **ชั้นแสดงผล/จัดเก็บ** (ResultView แสดงผลจากโครงสร้างผลลัพธ์มาตรฐาน, history.js บันทึกลง SQLite) ทำให้เปลี่ยนส่วนใดส่วนหนึ่งโดยไม่กระทบส่วนอื่น

2.4 แนวคิดการเขียนโปรแกรมเชิงวัตถุ (OOP)

2.4.1 Constructor

คลาสวิธีคำนวณทุกคลาสกำหนดข้อมูลเฉพาะของตัวเองไว้ใน constructor แล้วส่งต่อให้คลาสแม่ผ่าน super() ตัวอย่างคลาส BisectionMethod กำหนดว่าอินพุตของวิธีมีอะไรบ้าง (สมการ, ช่วง XL–XU, Tolerance, Max Iterations):

```
class BisectionMethod extends BaseMethod {
  constructor() {
    super({
      key: 'bisection', category: 'root',
      name: 'Bisection Method', nameTh: 'วิธีแบ่งครึ่งช่วง',
      fields: [
        { name: 'fx', label: 'f(x) = 0', type: 'expression' },
        { name: 'xl', label: 'XL (ปลายซ้าย)', type: 'number' },
        { name: 'xu', label: 'XU (ปลายขวา)', type: 'number' },
        { name: 'tol', label: 'Tolerance', type: 'number', default: 1e-6 },
        { name: 'maxIter', label: 'Max Iterations', type: 'int', default: 100 }
      ],
    });
  }
}
```

ข้อมูล fields นี้ถูกส่งต่อไปถึง Frontend ผ่าน API `GET /api/methods` เพื่อสร้างฟอร์มกรอกข้อมูลของแต่ละวิธีโดยอัตโนมัติ — เพิ่มวิธีใหม่ก็ได้ฟอร์มใหม่ทันทีโดยไม่ต้องแก้ Frontend

2.4.2 Encapsulation (การซ่อนรายละเอียด)

registry.js ซ่อนโครงสร้างเก็บวัตถุ (Map) ไว้ภายใน ภายนอกเรียกผ่านฟังก์ชัน register / getMethod / listMethods เท่านั้น และป้องกัน key ซ้ำตั้งแต่ตอนลงทะเบียน:

```
// registry.js — ซ่อน Map ไว้ข้างใน ภายนอกเข้าถึงผ่านฟังก์ชันเท่านั้น
const methods = new Map();

function register(method) {
  if (methods.has(method.meta.key)) {
    throw new Error(`method key ซ้ำ: ${method.meta.key}`);
  }
  methods.set(method.meta.key, method);
}

function getMethod(key) {
  return methods.get(key) || null;
}
```

นอกจากนี้ชั้น validation ยังทำหน้าที่เป็น "ด่านเดียว" ในการตรวจอินพุตทุกชนิด (จุดทศนิยะ, จำนวนเต็ม, ตารางจุด, เมทริกซ์, สมการ) โค้ดคำนวณจึงไม่ต้องตรวจข้อมูลเองซ้ำ ๆ อีก และ BaseMethod ซ่อนเงื่อนไขบังคับของ meta (ต้องมี key, category, name) ไว้ในตัว:

```
class BaseMethod {
  constructor(meta) {
    if (!meta || !meta.key || !meta.category || !meta.name) {
      throw new Error('method meta ต้องมี key, category และ name');
    }
    this.meta = meta;
  }
}
```

2.4.3 Composition (การประกอบวัตถุ)

registry.js ประกอบวัตถุวิธีคำนวณทั้ง 11 วัตถุเข้าเป็นระบบเดียว สังเกตว่าวิธีอนุพันธ์ทั้งสามตัว (Forward / Backward / Central) มาจากคลาสเดียวกัน คือ DifferenceMethod แต่ถูกสร้างเป็น 3 วัตถุด้วยค่า kind ต่างกัน:

```
[
  new BisectionMethod(),
  new NewtonRaphsonMethod(),
  new GaussEliminationMethod(),
  new LinearInterpolationMethod(),
  new LagrangeInterpolationMethod(),
  new LinearExtrapolationMethod(),
  new TrapezoidalRuleMethod(),
  new SimpsonsOneThirdRuleMethod(),
  new DifferenceMethod('forward'),
  new DifferenceMethod('backward'),
  new DifferenceMethod('central'),
].forEach(register);
```

แต่ละคลาสวิธีคำนวณยัง "ประกอบ" วัตถุอื่นเข้ามาใช้ เช่น ทุกคลาสประกอบวัตถุฟังก์ชัน f(x) ที่ได้จาก expression parser มาใช้คำนวณโดยไม่สนใจว่า parser เขียนอย่างไรภายใน

2.4.4 Polymorphism (พฤติกรรมหลากหลายรูปแบบ)

REST API ปลายทางเดียว `POST /api/calculate` ใช้คำนวณได้ทั้ง 11 วิธี — โค้ดเรียก `method.solve(input)` เพียงบรรทัดเดียว แต่พฤติกรรมจริงขึ้นอยู่กับว่า method เป็นวัตถุของคลาสใด (Bisection ก็แบ่งครึ่งช่วง, Gauss ก็ตัดตัวแปร):

```
// routes/index.js — โค้ดชุดเดียว ใช้กับวิธีใดก็ได้ (Polymorphism)
const method = getMethod(methodKey);
if (!method) {
  return res.status(400).json({ error: `ไม่พบวิธีคำนวณ '${methodKey}'` });
}
const parsed = {};   // ... ตรวจอินพุตตาม field.type ของวิธีนั้น ...
const result = method.solve(parsed);   // ← คลาสไหนก็เรียกเหมือนกัน
```

2.4.5 Abstraction และ Inheritance

BaseMethod เป็นคลาสนามธรรม: ประกาศเมธอด `solve()` ไว้ให้คลาสลูกสืบทอดและเขียนทับ (override) ตามสูตรของตัวเอง ใครลืม implement โปรแกรมจะโยน error ทันที:

```
// BaseMethod.js — เมธอดนามธรรม
/** @abstract */
solve(_input) {
  throw new Error(`วิธี '${this.meta.key}' ยังไม่ได้ implement เมธอด solve()`);
}

// roots.js — คลาสลูกเขียนทับด้วยอัลกอริทึมของตัวเอง
class NewtonRaphsonMethod extends BaseMethod {
  solve({ fx, x0, tol, maxIter }) {
    // x_{i+1} = x_i − f(x_i) / f'(x_i) ...
  }
}
```

คลาสลูกทุกตัวได้คุณสมบัติร่วมจากคลาสแม่โดยอัตโนมัติ: โครงสร้าง meta แบบเดียวกัน, การถูกเก็บใน registry, และรูปแบบผลลัพธ์มาตรฐาน { answer, steps, table, meta } ที่ ResultView นำไปแสดงผลได้ทุกวิธีโดยไม่ต้องมีเงื่อนไขแยกวิธี

2.5 อัลกอริทึมที่สำคัญ

2.5.1 Expression Parser (ไม่ใช้ eval)

เพื่อความปลอดภัย โปรแกรมแยกวิเคราะห์สมการ f(x) จากสตริงด้วย Tokenizer + Recursive Descent Parser ที่เขียนขึ้นเอง รองรับตัวดำเนินการ + – * / ^ วงเล็บ ฟังก์ชัน sin cos tan sqrt exp ln log abs ค่าคงที่ pi, e และการคูณละเครื่องหมาย (2x, 3(x+1)):

```
function tokenize(src) {
  const tokens = [];
  let i = 0;
  while (i < src.length) {
    const ch = src[i];
    if (/[0-9.]/.test(ch)) {          // ตัวเลข (รวมเลขยกกำลัง 1e-6)
      const m = /^(?:\d+\.?\d*|\.\d+)(?:[eE][+-]?\d+)?/.exec(src.slice(i));
      tokens.push({ type: 'num', value: parseFloat(m[0]) });
      i += m[0].length;
    } else if (/[a-zA-Z_]/.test(ch)) { // ตัวแปร / ฟังก์ชัน / ค่าคงที่
      ...
    } else if ('+-*/^(),'.includes(ch)) {
      tokens.push({ type: ch });
      i++;
    } else {
      throw new ExprError(`ไม่รู้จักอักขระ '${ch}' ที่ตำแหน่งที่ ${i+1}`);
    }
  }
  return tokens;
}
```

2.5.2 Bisection Method — การทำซ้ำแบบแบ่งครึ่งช่วง

```
solve({ fx, xl, xu, tol, maxIter }) {
  if (xl >= xu) throw ...;
  const fxl = evalAt(fx, xl, 'f(XL)');
  const fxu = evalAt(fx, xu, 'f(XU)');
  if (fxl * fxu > 0) {
    throw ...;   // f(XL), f(XU) เครื่องหมายเดียวกัน → ใช้ Bisection ไม่ได้
  }
  let a = xl, b = xu, prev = null, root;
  for (let iter = 0; iter < maxIter; iter++) {
    root = (a + b) / 2;                          // จุดกึ่งกลาง
    const froot = evalAt(fx, root, 'f(x)');
    if (fa * froot < 0) b = root; else a = root; // เลือกครึ่งที่มีราก
    if (prev !== null && Math.abs((root - prev) / root) * 100 < tol) break;
    prev = root;
  }
}
```

ทุกรอบการทำซ้ำถูกเก็บลงตาราง (XL, XU, XR, f(XL), f(XR), Error %) เพื่อแสดงให้ผู้ใช้เห็นการลู่เข้าของคำตอบทีละรอบ

2.5.3 Gauss Elimination — การตัดตัวแปรแบบ Partial Pivoting

```
for (let col = 0; col < n - 1; col++) {
  // เลือกแถวที่มีค่าสัมบูรณ์มากที่สุดในคอลัมน์นี้เป็น pivot (ลดปัญหาตัวเลข)
  let maxRow = col;
  for (let r = col + 1; r < n; r++) {
    if (Math.abs(A[r][col]) > Math.abs(A[maxRow][col])) maxRow = r;
  }
  [A[col], A[maxRow]] = [A[maxRow], A[col]];
  [b[col], b[maxRow]] = [b[maxRow], b[col]];

  if (Math.abs(A[col][col]) < 1e-12) {
    throw ...;   // เมทริกซ์เอกฐาน (singular) — แก้ระบบไม่ได้
  }
  for (let r = col + 1; r < n; r++) {  // ตัดตัวแปรแถวล่าง
    const m = A[r][col] / A[col][col];
    for (let c = col; c < n; c++) A[r][c] -= m * A[col][c];
    b[r] -= m * b[col];
  }
}
// แทนค่าย้อนกลับ (Back Substitution)
```

2.5.4 Newton-Raphson — อนุพันธ์เชิงตัวเลข

```
// หาอนุพันธ์ด้วย central difference ความแม่นยำสูง
const df = (x) => {
  const h = Math.max(1e-9, Math.abs(x) * 1e-8);  // h แปรผันตามขนาด x
  return (evalAt(fx, x + h, ...) - evalAt(fx, x - h, ...)) / (2 * h);
};
// ทำซ้ำ: x ใหม่ = x − f(x) / f'(x)  ถ้า f'(x) = 0 → แจ้งเตือนหารด้วยศูนย์
```

บทที่ 3
สรุปผลการพัฒนาโครงงาน

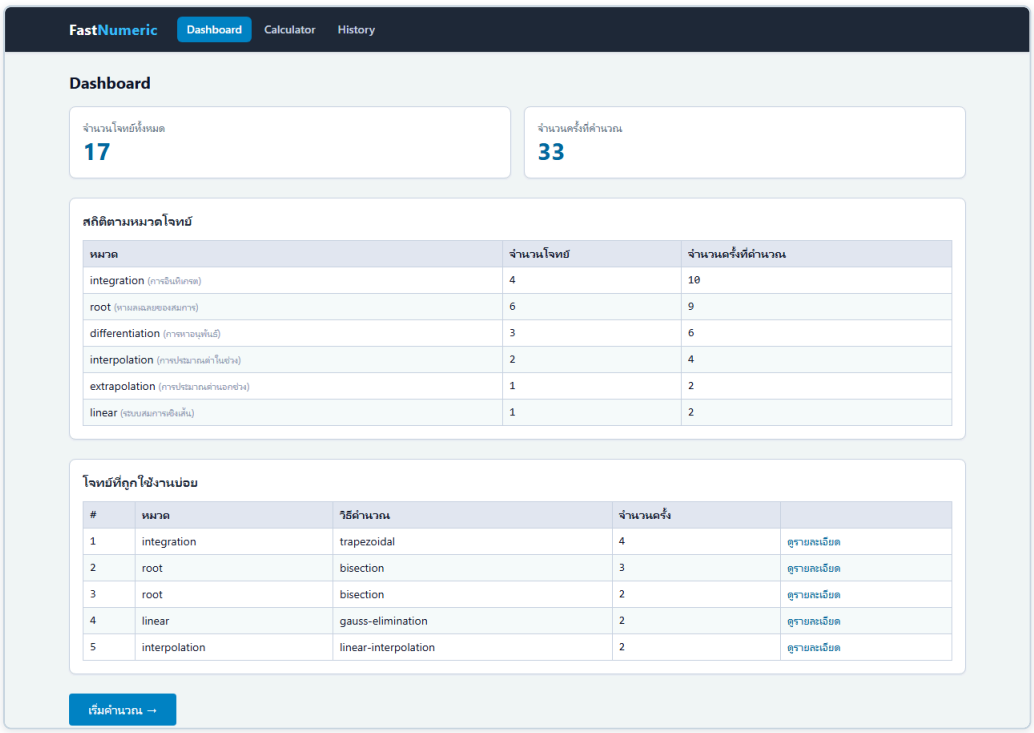
3.1 ปัญหาที่พบระหว่างการพัฒนา

ปัญหา	แนวทางแก้ไข
การใช้ eval() กับสมการ f(x) เสี่ยงต่อความปลอดภัยและข้อความ error อ่านไม่เข้าใจ	เขียน Expression Parser เอง (Tokenizer + Recursive Descent) แปลง error เป็นข้อความไทยระบุตำแหน่งอักขระที่ผิด เช่น "ไม่รู้จักอักขระ '@' ที่ตำแหน่งที่ 3"
การกรอกเมทริกซ์ฝั่งผู้ใช้ส่งชื่อ key ไม่ตรงกับฝั่งเซิร์ฟเวอร์ (A พิมพ์ใหญ่/เล็ก) ทำให้ได้ข้อความ "รูปแบบข้อมูลเมทริกซ์ไม่ถูกต้อง"	ปรับ asMatrix ให้รับทั้ง key "a" และ "A" พร้อมเขียนชุดทดสอบ API ครอบคลุมกรณีนี้
หน้าเว็บขึ้น "Cannot read properties of null (reading 'categories')" เมื่อ API ยังไม่ตอบ	ตั้ง Vite dev proxy ให้ /api ไปยังเซิร์ฟเวอร์ Express และเพิ่มการตรวจสอบ response ก่อน setState (Array.isArray(d.categories))
วิธี Simpson ต้องการจำนวนช่วงเป็นเลขคู่ แต่ผู้ใช้กรอกเลขคี่ได้	ตรวจที่ด่าน validation แล้วแจ้งข้อความไทยพร้อมคำแนะนำให้กรอกใหม่
Newton-Raphson ไม่ลู่เข้าเมื่อ f'(x) = 0 หรือเริ่มที่จุดที่ไม่เหมาะสม	ไม่แอบอ้างว่าคำตอบถูก — แสดงค่าประมาณพร้อมแถบเตือนสีเหลืองอธิบายสาเหตุและแนะนำให้เปลี่ยน X0
โจทย์เดิมถูกบันทึกซ้ำ ๆ ทำให้ตารางประวัติรก	ใช้ SHA-256 hash ของ (หมวด + วิธี + อินพุตที่เรียง key แล้ว) เป็น UNIQUE constraint โจทย์เดิมจะเพิ่มแค่ "จำนวนครั้งที่คำนวณ"

3.2 จุดเด่นของโปรแกรม

3.2.1 หน้า Dashboard สรุปสถิติแบบเรียลไทม์

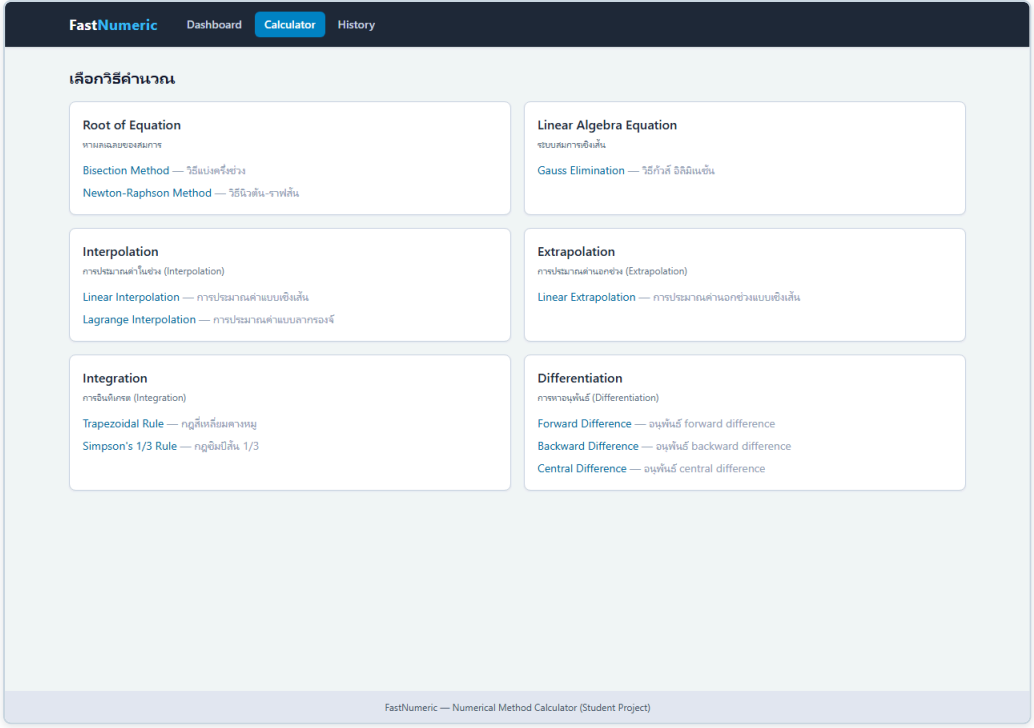
แสดงจำนวนโจทย์ทั้งหมด จำนวนครั้งที่คำนวณ สัดส่วนตามหมวด และโจทย์ยอดนิยม ข้อมูลทั้งหมดอ่านจาก SQLite จริงที่ผู้ใช้คำนวณเอง



รูปที่ 3.1 หน้า Dashboard — สถิติการใช้งานและโจทย์ยอดนิยม

3.2.2 เลือกวิธีคำนวณได้ 6 หมวด 11 วิธี

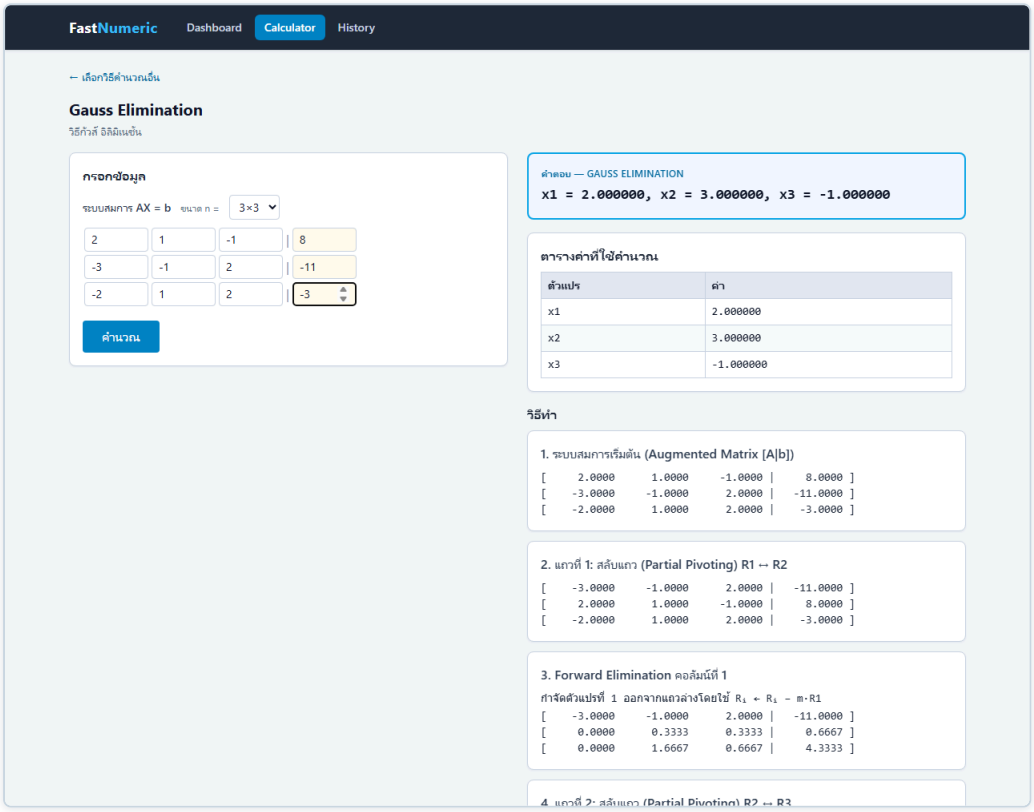
หน้า Calculator จัดกลุ่มวิธีคำนวณตามหมวด รายการวิธีทั้งหมดอ่านจาก API โดยตรง (ไม่มีรายการแข็งในโค้ด) เพิ่มวิธีใหม่ที่ฝั่งเซิร์ฟเวอร์แล้วหน้าเว็บแสดงเพิ่มทันที



รูปที่ 3.2 หน้าเลือกวิธีคำนวณ — 6 หมวด 11 วิธี

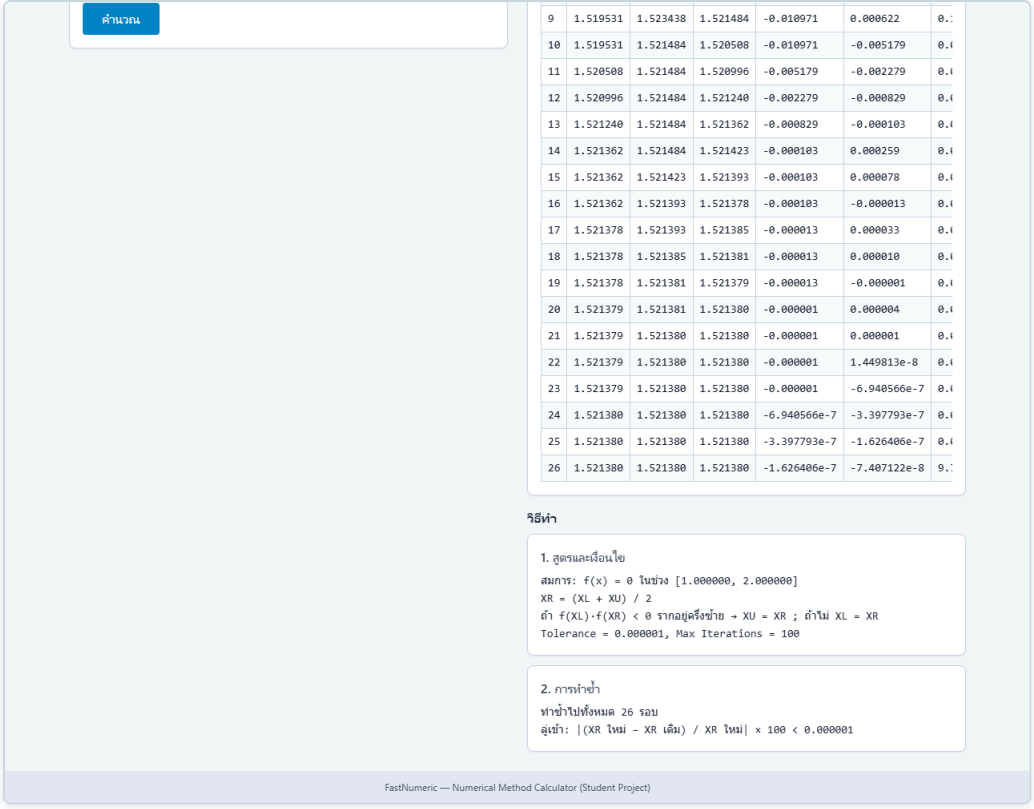
3.2.3 แสดงขั้นตอนและตารางการทำซ้ำครบทุกวิธี

ผลลัพธ์แต่ละครั้งแสดง "คำตอบ", "ตารางการทำซ้ำ" และ "วิธีทำ" แบบทีละขั้น ผู้ใช้ตรวจการคำนวณย้อนกลับได้ เช่น แก้ระบบสมการด้วย Gauss Elimination จะเห็น Augmented Matrix ตั้งต้น การสลับแถวแบบ Partial Pivoting และการตัดตัวแปรทีละขั้น



รูปที่ 3.3 ผลลัพธ์ Gauss Elimination — คำตอบ x1=2, x2=3, x3=−1 พร้อมวิธีทำ

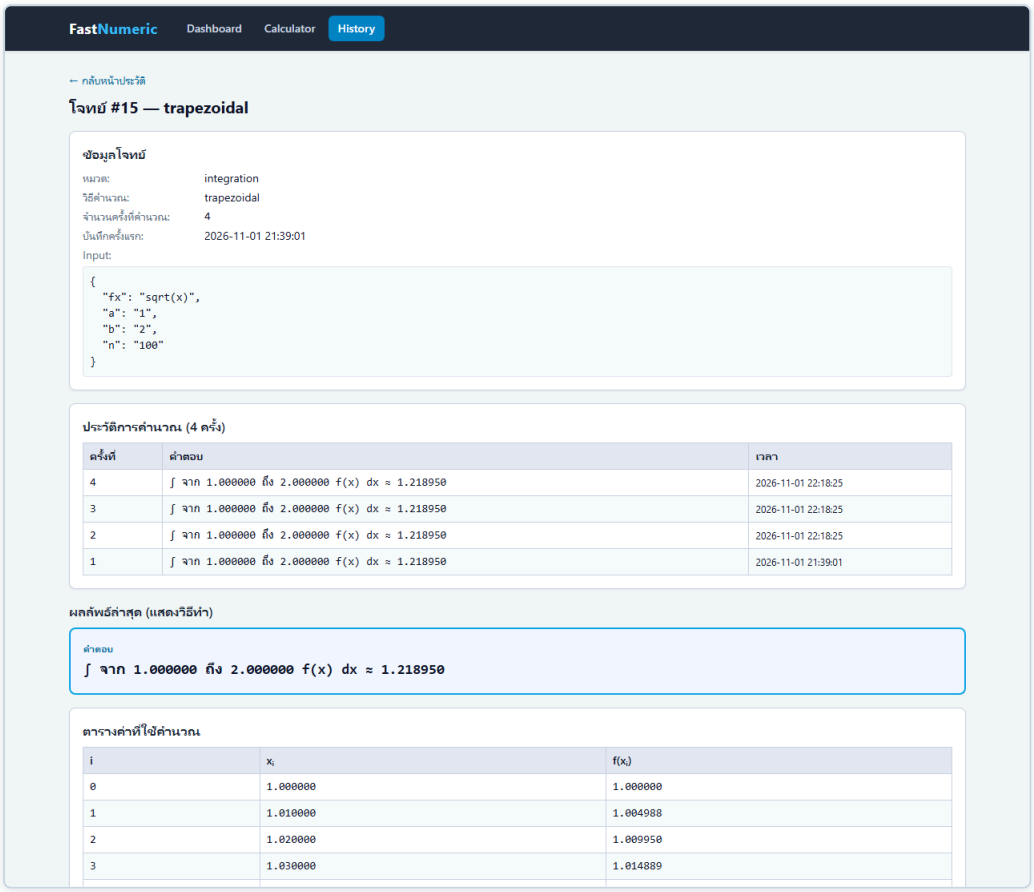
กรณีหารากสมการด้วย Bisection จะได้ตารางการทำซ้ำครบทุกรอบพร้อมค่า error ที่ลดลงเรื่อย ๆ จนบรรลุ Tolerance:



รูปที่ 3.4 ตารางการทำซ้ำ Bisection 26 รอบ ลู่เข้าหาราก ≈ 1.521380

3.2.4 ระบบประวัติการคำนวณพร้อมนับโจทย์ซ้ำ

ทุกการคำนวณถูกบันทึกลง SQLite โจทย์ที่คำนวณซ้ำ (หมวด + วิธี + อินพุตเดิม) จะไม่ถูกบันทึกซ้ำแต่เพิ่ม "จำนวนครั้งที่คำนวณ" ให้อัตโนมัติ สามารถเปิดดูรายละเอียดและลบรายการได้



รูปที่ 3.5 รายละเอียดโจทย์ — แสดงจำนวนครั้งที่คำนวณและผลลัพธ์ล่าสุด

3.3 การทดสอบระบบ

ระบบผ่านการทดสอบดังนี้: ชุดทดสอบหน่วย (unit test) ของอัลกอริทึมทั้ง 37 กรณี ผ่านทั้งหมด; ทดสอบ API ทั้ง 11 วิธีด้วยโจทย์ที่รู้คำตอบจริง (เช่น Bisection/Newton ของ x^3-x-2 ได้ราก ≈ 1.521380, Gauss ระบบ 3×3 ได้ x = (2, 3, −1), Simpson ของ x^2 บน [0,1] ได้ 1/3 แม่นสุดทศนิยม) ผ่านทั้งหมด; ทดสอบกรณี error 14 กรณี (หารด้วยศูนย์, เมทริกซ์เอกฐน, เครื่องหมายช่วงผิด, x ซ้ำ, จำนวนช่วงเลขคี่ ฯลฯ) ระบบแจ้งข้อความไทยถูกต้องทุกกรณี; และทดสอบการนับโจทย์ซ้ำ การคงอยู่ของข้อมูลหลังรีสตาร์ทเซิร์ฟเวอร์ ผ่านครบ

3.4 คำแนะนำในการพัฒนาต่อ

1. เพิ่มการวาดกราฟ f(x) พร้อมประมาณการตำแหน่งราก เพื่อช่วยเลือกช่วง XL–XU ที่เหมาะสม
2. เพิ่มวิธีคำนวณขั้นสูง เช่น Secant Method, Gauss-Jordan, Newton's Divided Difference Interpolation ซึ่งทำได้ง่ายจากโครงสร้างคลาสเดิม (สร้างคลาสใหม่ extends BaseMethod แล้ว register)
3. เพิ่มการล็อกอินผู้ใช้ เพื่อแยกประวัติการคำนวณของแต่ละคน
4. ส่งออกผลลัพธ์เป็น PDF หรือรูปภาพ เพื่อแนบในรายงานของผู้ใช้ได้โดยตรง